

Evaluation Comments Template

General Information

- **Evaluator Name:** P1 _____
 - **Application Name:** spring-petclinic _____
 - **Refactoring Approach:** monomorph _____
-

1) Services Boundaries and Decompositions

1. Are all of the classes in the application mapped to their correct microservices based on their decomposition?

Yes. All classes were mapped to their proper service.

2. Are there classes that were not included in the decomposition but found in the monolith? How were they handled by the approach?

Describe any missing or unaccounted classes:

All classes were included in the decomposition.

3. Are there new classes added to the microservices? Do they still respect the single responsibility of these microservices?

Yes. The approach added DTOs and other boilerplate code for ID-based calls as well as protobuf-generated objects. The added classes still respect the single responsibility principle.

4. Has the dependency mapping between components been accurately preserved?

Yes.

5. How well did the approach handle this aspect? (Score from 1 to 10)

10/10.

6. Additional Comments

2) Correctness & Functional Equivalence:

1. Does the refactored application produce the same outcomes as the original monolith?

Human intervention would be needed to fix transformation issues.

2. Have any functional behaviors been inadvertently altered during the transformation?

While it looks like functional behaviors have not been altered. The added complexity and requirement for human intervention to fix issues beforehand makes it hard to guarantee

3. Is the transformation strictly limited to modifying call mechanics, leaving the business logic unchanged?

Yes. Generated DTOs and remote calls are transparent from the business logic point of view.

4. Do the transformed interactions maintain the same logical flow as in the monolith?

5. How well did the approach handle this aspect? (Score from 1 to 10)

6/10.

6. Additional Comments

3) Code Transformation and Remote Invocations:

1. Are all relevant local method calls successfully replaced by their remote call counterparts?

Local calls were replaced by remote calls for the most part, although some issues due to non-replaced imports and broken code generation prevented successful replacement of all calls.

2. Are the request and response structures consistent with the original local calls?

Yes. Generated DTOs and responses match the structures expected from local calls.

3. Are there any issues such as missing object mappings or incorrect handling of requests and responses?

As mentioned, there is multiple syntactic issues in the generated code and missed replacement steps for some imports such as the “Visit” type between the “VisitController” and “Pet” client which creates a conflict between the original “Visit” item and the generated “Visit”.

4. Is data correctly serialized and deserialized during remote calls?

Yes. Mappers are used to convert to and from DTOs.

5. How well did the approach handle this aspect? (Score from 1 to 10)

6/10

6. Additional Comments

Mentioned issues with generated code are, for instance, incorrect block comment usage in the customers service’s “server/PetImpl” or missing closing parenthesis in the visits service’s “client/Owner”.

4) Data Handling and Consistency:

1. Is there an existing database in the monolith? How did the refactoring approach handle it?

The monolith uses Spring Data repositories to handle persistence and is preserved as expected in the refactored services.

2. How is a shared state managed or synchronized across the services?

All persistent entities are own by the proper services and exposed through endpoints.

3. Is there any data duplication? Do you think that it is justified?

No data duplication.

4. Is the data still consistent? Were there any new consistency mechanisms added if needed?

No new mechanisms added.

5. How well did the approach handle this aspect? (Score from 1 to 10)

10/10.

6. Additional Comments

5) Performance Overhead:

1. Are there new mechanisms introduced that add overhead to existing interactions?

Yes. The approach adds an ID-based approach as well as DTOs, based on gRPC. The generated DTO clients also encapsulate its own remote calls when inter-services communication is necessary.

2. Do you think that this overhead is justified?

Somewhat. The added complexity adds a significant learning curve and lacks transparency compared to the original code in my opinion. But there is definitely value in having all interactions, whatever their level of complexity is, be handled in the same way as it makes it easier to understand and use overtime.

3. What mechanisms would you replace if you had to modify the code manually?

One change I might look into is to hide the gRPC fields and calls from the generated clients and servers so it's easier to compare and edit them post-transformation.

4. Are there any indications of over-segmentation leading to excessive inter-service communication?

5. How well did the approach handle this aspect? (Score from 1 to 10)

8/10.

6. Additional Comments

6) Code Maintainability & Readability Post-Transformation:

1. Is the refactored code easy to understand and follow compared to the original monolith?

Refactored services are easy to read for the most part. Original code is kept as is. For this application, the complexity of the added mechanisms is on the edge of being too much and still definitely makes for the most difficult part to read and understand.

2. Has the transformation introduced excessive boilerplate code or complexity?

Not excessive. The transformation added a lot of complexity, but redeems itself by providing a robust framework from which to work from.

3. How difficult is it to replace the mechanisms introduced by the approach if needed?

Difficult. Although replacing the mechanisms would be a faster than rewriting the whole thing granted the developers took the time to familiarize themselves with said mechanisms beforehand.

4. Does the approach include fault tolerance or error handling mechanisms?

Some minimal error handling is present through checks such as “response.hasPet”.

5. How well did the approach handle this aspect? (Score from 1 to 10)

7/10.

6. Additional Comments

7) Execution and Packaging:

1. Does the refactored version compile and/or package correctly?

No. The transformed code contains multiple issues such as missing imports and broken code generation that prevents compilation.

2. Does the refactoring approach introduce new dependencies?

Yes. It adds gRPC and protobuf dependencies, as well as a caching library.

3. Does the approach introduce any new deployment automation mechanisms?

No. It also added the new dependencies to the existing maven project, but skipped Gradle.

4. How well did the approach handle this aspect? (Score from 1 to 10)

6/10.

5. Additional Comments

8) Additional comments that do not fall into any aspect:
